

No What Already Works

- WebRTC-based live video streaming from webcam or device to the server
- Real-time WebSocket communication pushing data to the frontend
- Emotion detection using the FER library, triggered periodically on video frames
- Food recognition via Clarifai image classification and Foodvisor API
- Basic intolerance checking using simple text matching against a user-provided list
- Nutritional information retrieval through Azure OpenAI (GPT-4)
- Video recording with webm-to-mp4 conversion via ffmpeg
- MongoDB storage for user records

Gap Analysis

Requirement	Status	Notes
Child detection	Not started	No person/child detection in pipeline
Cough detection	Not started	No audio processing exists
Sneeze detection	Not started	No audio processing exists
Emotion recognition	Partial	Works but blocks event loop; needs optimisation
Food identification	Partial	Clarifai + Foodvisor in place; accuracy can improve
Estimated quantity eaten	Not started	No plate comparison or portion tracking
Child status logging	Not started	No structured logs for events
Food diary logging	Not started	No meal session or nutrition logs
Allergen management	Basic	Simple substring match; no per-child profiles

Online dashboard	Not started	Current UI is a minimal process page
------------------	-------------	--------------------------------------

Phase Breakdown

Phase 1: Code Restructuring & Data Model Design

Objective: Transform the single-file application into a modular architecture and design the database schema that will support all required logging.

Key Activities

1. Separate the existing code into distinct modules: routing, WebRTC handling, WebSocket management, ML services, and database operations.
2. Remove global state variables and replace them with proper session management tied to individual connections.
3. Design MongoDB collections for children (profiles, allergies), meal sessions, food diary entries, child status events, and allergen logs.
4. Create data validation models to ensure consistent data quality across all logging.
5. Migrate any existing data from the current userLists collection to the new schema.

Outcome

A clean, modular codebase with a robust database foundation. All subsequent phases will build on top of this structure without requiring further architectural changes.

Phase 2: Child Detection

Objective: Enable the system to detect the presence of a child in the video frame and track their position throughout the meal.

Key Activities

1. Integrate a pre-trained object detection model (such as YOLOv8) into the video processing pipeline.
2. Configure the model to detect persons, using size and proportion heuristics to distinguish children from adults where possible.
3. Run detection at a balanced interval (e.g. every 15 frames) to maintain real-time performance without overloading the server.
4. Log child presence and absence events to the child status collection with timestamps.

5. Push detection status to the frontend in real time via the existing WebSocket channel.

Outcome

The system can confirm a child is present and seated during a meal session. This detection feeds into other features (emotions are only logged when a child is detected, food tracking is tied to a confirmed session).

Phase 3: Cough & Sneeze Detection

Objective: Monitor the audio stream for cough and sneeze events, logging each occurrence as a child health status event.

Key Activities

1. Extract audio from the WebRTC stream and buffer it into short analysis windows (1–2 seconds).
2. Integrate a pre-trained audio event classifier (such as YAMNet) that can recognise cough and sneeze sounds among hundreds of audio event categories.
3. Set confidence thresholds and implement debouncing (e.g. a 3-second cooldown) to avoid duplicate logging of a single cough or sneeze.
4. Log each detected event to the child status collection with event type, timestamp, and confidence score.
5. Send real-time alerts to the frontend when a cough or sneeze is detected.

Outcome

Health-related audio events are captured automatically during every meal session, building a longitudinal record of coughing and sneezing patterns for each child.

Phase 4: Emotion Detection Enhancement

Objective: Improve the existing emotion detection for reliability and richer data, and ensure it is properly logged.

Key Activities

1. Move the emotion detection computation off the main event loop to prevent it from blocking video frame delivery.

2. Evaluate whether the current FER library provides sufficient accuracy for children's expressions, and assess alternative models if needed.
3. Store the full emotion distribution (all emotion scores) alongside the dominant emotion for each detection.
4. Generate per-session emotion summaries (e.g. percentage of time spent in each emotional state) at the end of each meal.
5. Ensure emotion events are only logged when child detection confirms a child is present in the frame.

Outcome

Emotion data is captured reliably without impacting video performance, and stored in a structured format that supports both real-time display and historical dashboard analysis.

Phase 5: Food Recognition & Quantity Estimation

Objective: Identify foods on the plate, estimate how much has been eaten, and calculate nutritional values.

Key Activities

1. Enhance the food recognition pipeline by cross-referencing Clarifai classification results with Foodvisor's detailed food analysis API.
2. Consider adding a vision-language model as a fallback for ambiguous food items where traditional classifiers have low confidence.
3. Implement quantity estimation by comparing plate snapshots at the start and end of the meal, using image analysis to approximate the percentage of food consumed.
4. Replace the current free-text nutritional lookup with a structured nutrition database or API that returns consistent, machine-readable data (calories, macronutrients, micronutrients).
5. Record meal start time, end time, and total duration automatically based on processing start/stop signals.
6. Log all food data to the food diary collection, including foods identified, nutritional values, estimated calories, quantity eaten, and food remaining.

Outcome

Each meal session produces a complete food diary entry with identified foods, nutritional

breakdown, estimated consumption, and meal timing — all stored in the database and available for the dashboard.

Phase 6: Allergen Management System

Objective: Build a comprehensive, per-child allergen tracking system that automatically checks detected foods against known allergies.

Key Activities

1. Create a master allergen list pre-populated with the major regulatory allergens, with the ability for users to add custom allergens.
2. Build per-child allergy profiles where each child's specific allergies are selected from the master list via a tick/checkbox interface.
3. Replace the current simple substring matching with a proper food-to-allergen lookup — either via a food ingredient database, a dedicated API, or an AI-assisted allergen analysis.
4. When a food is detected during a meal, automatically check it against the active child's allergy profile and log whether an allergen was detected or not.
5. Send immediate real-time alerts via WebSocket if a potential allergen is identified in the food being served.

Outcome

Every meal is automatically screened against the child's known allergies. The system logs all allergen checks (both detections and clearances) to provide a full audit trail in the dashboard.

Phase 7: Online Dashboard

Objective: Deliver a multi-tab web dashboard that presents all logged data in an accessible, filterable format.

Key Activities

1. Build a set of REST API endpoints that serve structured data from the database to the frontend: session summaries, food diary entries, allergen logs, and child status events.
2. Develop the Dashboard (overview) tab showing summary statistics, recent activity, and live session status — styled similarly to the previous Mealtimes with Cammy interface.
3. Develop the Food Diary Log tab with a filterable table of meal sessions showing foods, nutrition, calories, and meal duration.

4. Develop the Allergens Log tab with a filterable table of allergen detection events, showing food, allergen type, and alert status.
5. Develop the Child Status Log tab with a filterable table of cough, sneeze, emotion, and temperature events, including a timeline view.
6. Apply the client's UI designs when provided. Build with a clean component structure so that visual styling can be changed independently of functionality.

Outcome

A fully functional online dashboard where caregivers can review meal history, monitor health events, check allergen safety, and track child wellbeing over time.

Phase 8: User-Entered Data & Configuration

Objective: Allow users to input and manage data that the system cannot yet detect automatically.

Key Activities

- Build a child profile management page where users can add or edit child details: name, age, sex, and allergy selections.
- Add a body temperature input field (manual entry for the initial release, with the expectation that T40 device integration will automate this in a future iteration).
- Implement a session setup flow where the user selects the child and confirms the device before starting a meal monitoring session.
- Create a settings area for managing the master allergen list, adjusting detection sensitivity thresholds, and configuring notification preferences.
- Assign device location labels so that all logs automatically include where the monitoring took place.

Outcome

The system captures all required contextual data (child details, location, temperature) through user input, ensuring every log entry is complete and meaningful even before hardware-based automation is available.